

Session11_10.07.2026

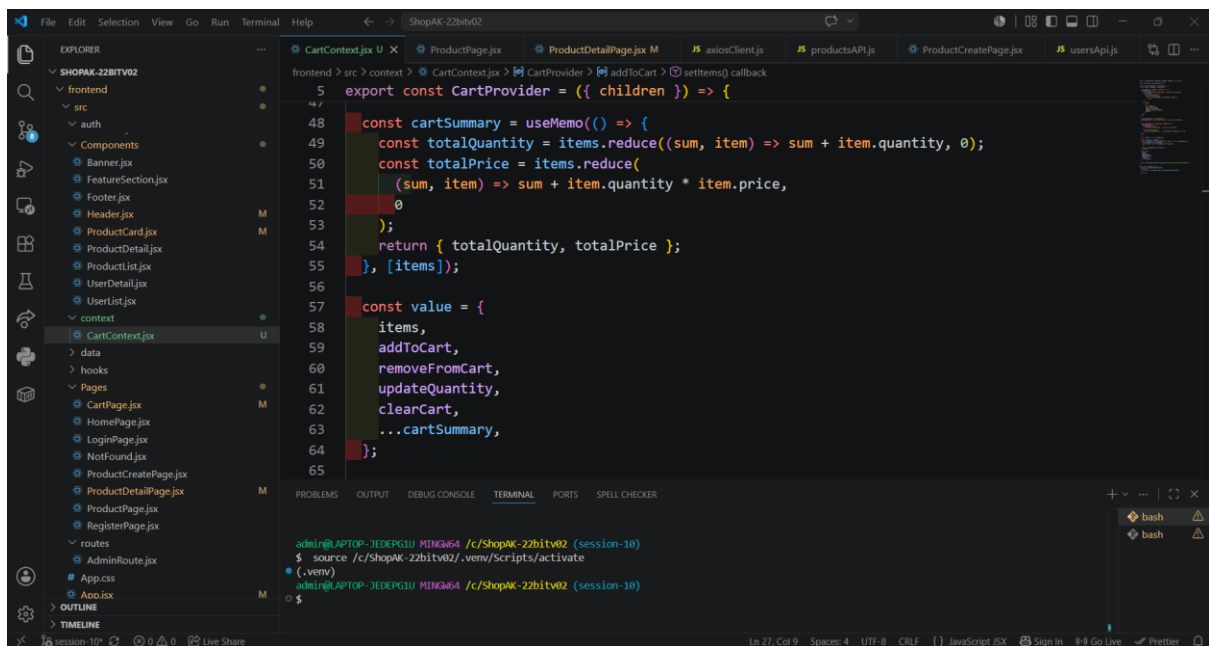
Lab Tasks

1. Context Setup

Tạo CartContext và CartProvider với:

- items array.
- addToCart(product, quantity).
- removeFromCart(productId).
- updateQuantity(productId, newQuantity).
- clearCart().

Tính totalQuantity và totalPrice bằng reduce và chia sẻ trong context.

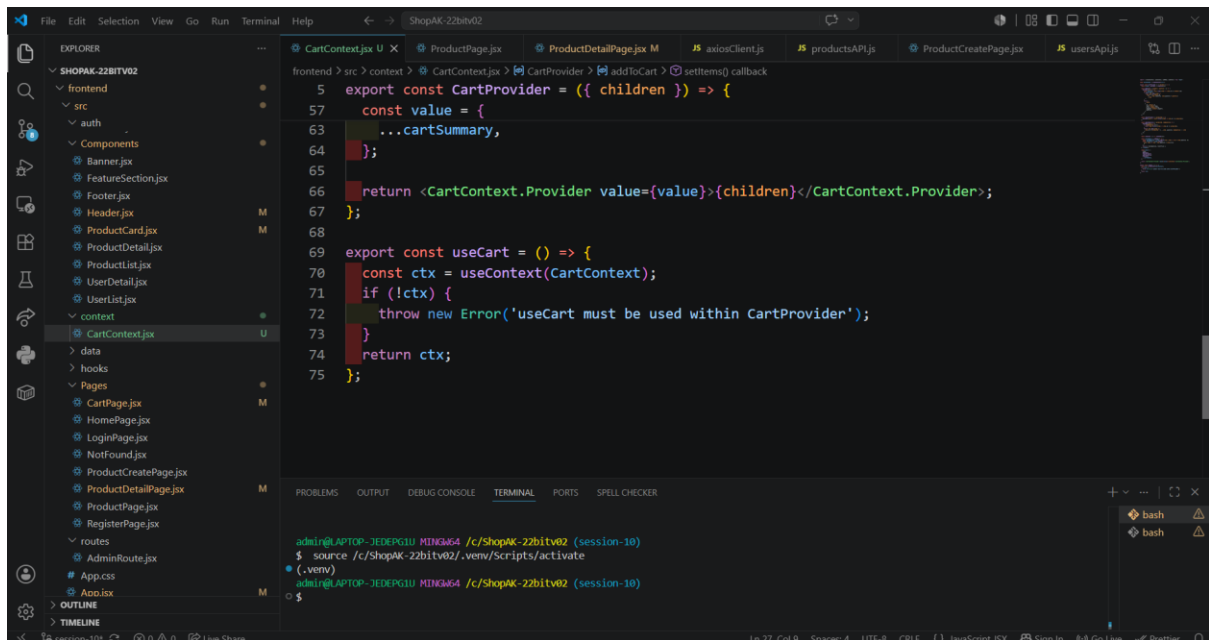


The screenshot shows a VS Code editor with a project named 'ShopAK-22bitv02'. The file explorer on the left shows a directory structure with 'frontend' containing 'src' and 'context'. The 'context' directory contains 'CartContext.jsx'. The main editor shows the code for 'CartContext.jsx' and 'CartProvider'.

```
5 export const CartProvider = ({ children }) => {
48   const cartSummary = useMemo(() => {
49     const totalQuantity = items.reduce((sum, item) => sum + item.quantity, 0);
50     const totalPrice = items.reduce(
51       (sum, item) => sum + item.quantity * item.price,
52       0
53     );
54     return { totalQuantity, totalPrice };
55   }, [items]);
56   const value = {
57     items,
58     addToCart,
59     removeFromCart,
60     updateQuantity,
61     clearCart,
62     ...cartSummary,
63   };
64   };
65 }
```

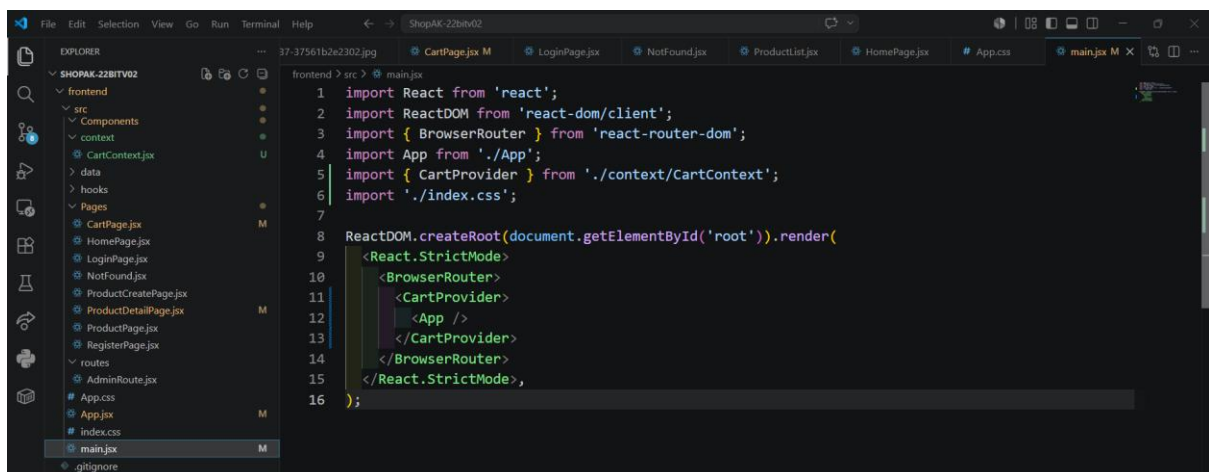
The terminal at the bottom shows the command prompt with the following commands and output:

```
admin@LAPTOP-JEDEPG1U MINGW64 /c/ShopAK-22bitv02 (session-10)
$ source ./scripts/activate
(.venv)
admin@LAPTOP-JEDEPG1U MINGW64 /c/ShopAK-22bitv02 (session-10)
$
```

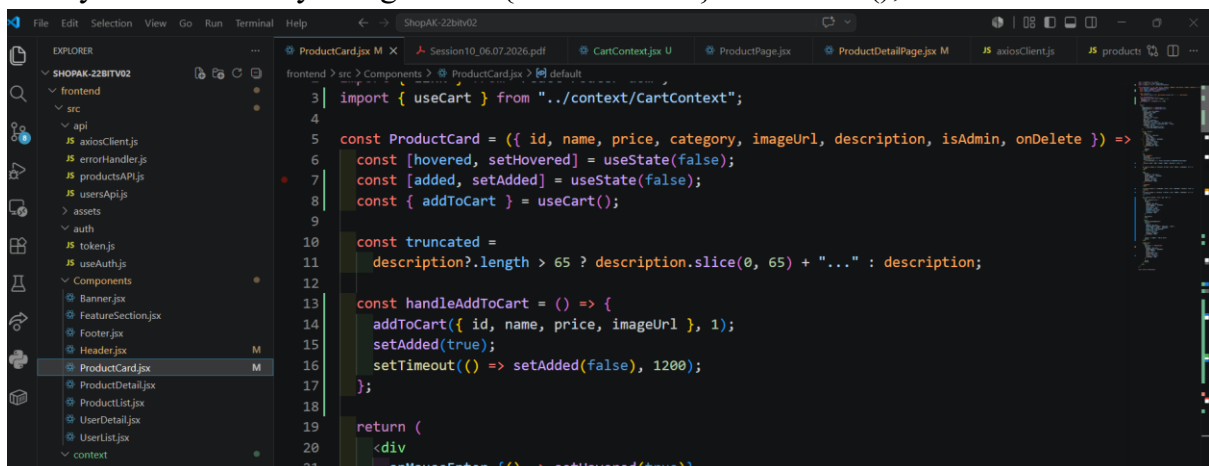


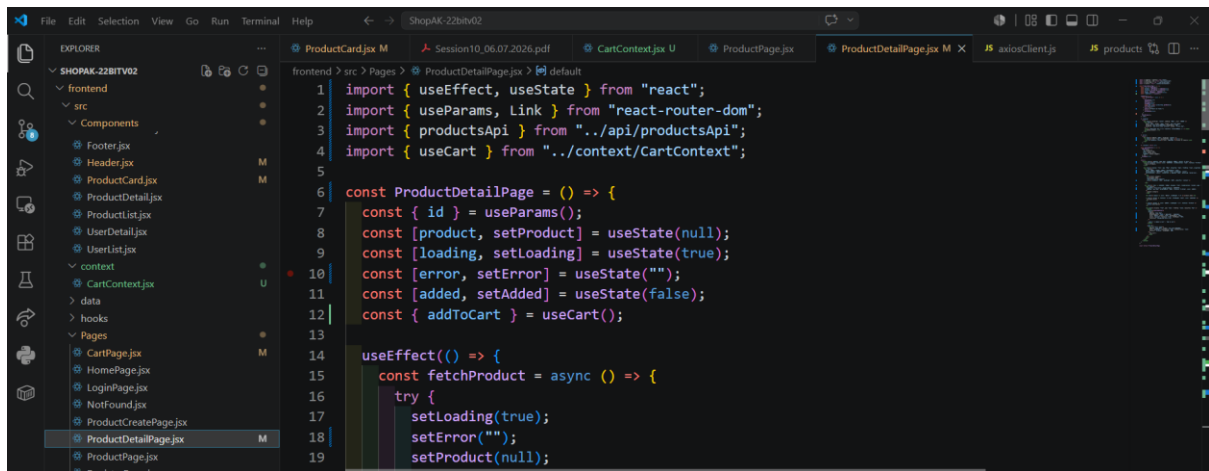
2. Tính totalQuantity và totalPrice bằng reduce và chia sẻ trong context

Bọc App bằng CartProvider



Ở đây ta có thể thấy dòng `const { AddToCart } = useCart();` và `handleAddToCart`

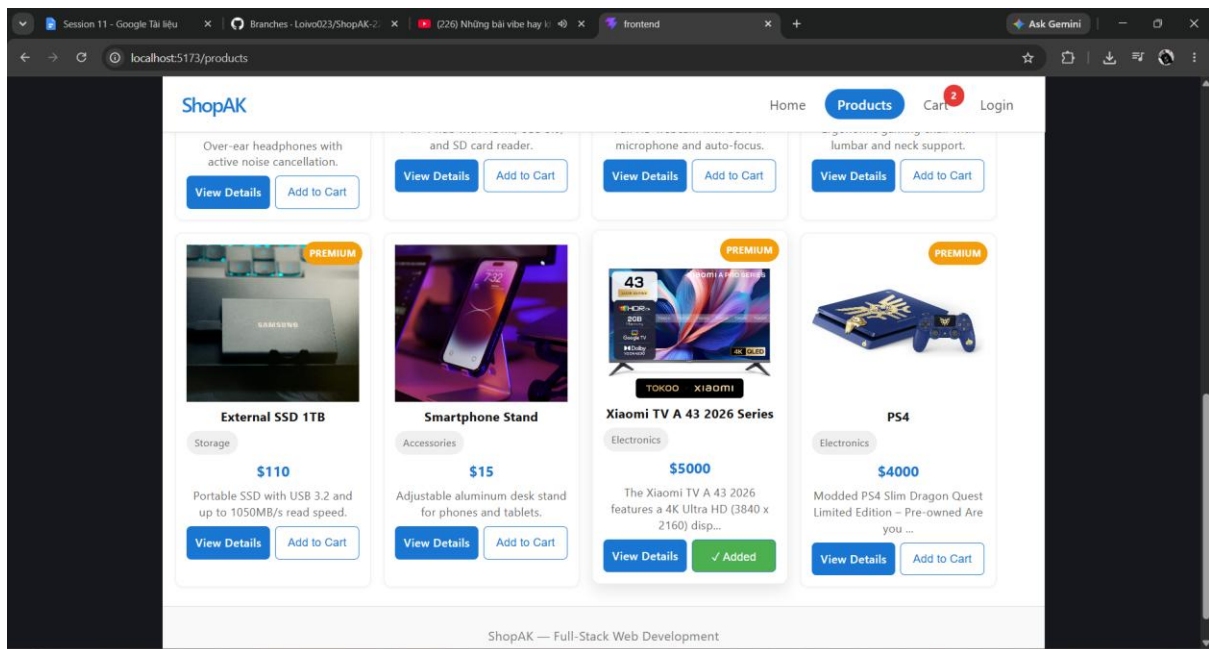




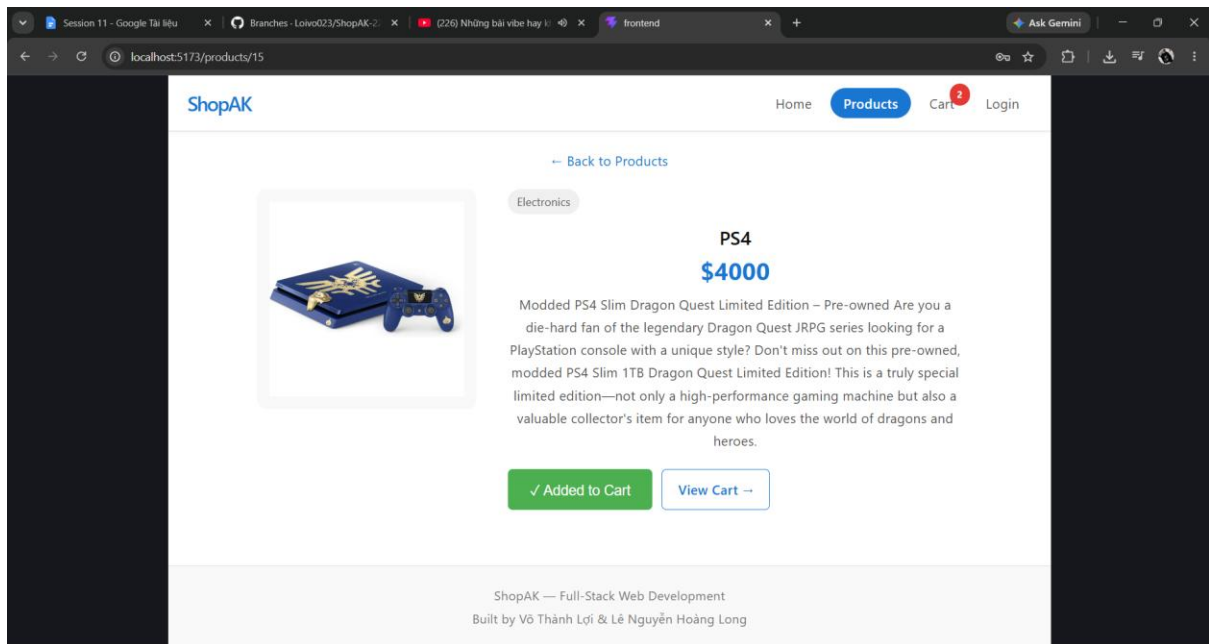
```
1 import { useEffect, useState } from "react";
2 import { useParams, Link } from "react-router-dom";
3 import { productsApi } from "../api/productsApi";
4 import { useCart } from "../context/CartContext";
5
6 const ProductDetailPage = () => {
7   const { id } = useParams();
8   const [product, setProduct] = useState(null);
9   const [loading, setLoading] = useState(true);
10  const [error, setError] = useState("");
11  const [added, setAdded] = useState(false);
12  const { addToCart } = useCart();
13
14  useEffect(() => {
15    const fetchProduct = async () => {
16      try {
17        setLoading(true);
18        setError("");
19        setProduct(null);
```

Ở ProductCard và ProductDetailPage, dùng addToCart khi click “Add to Cart”.

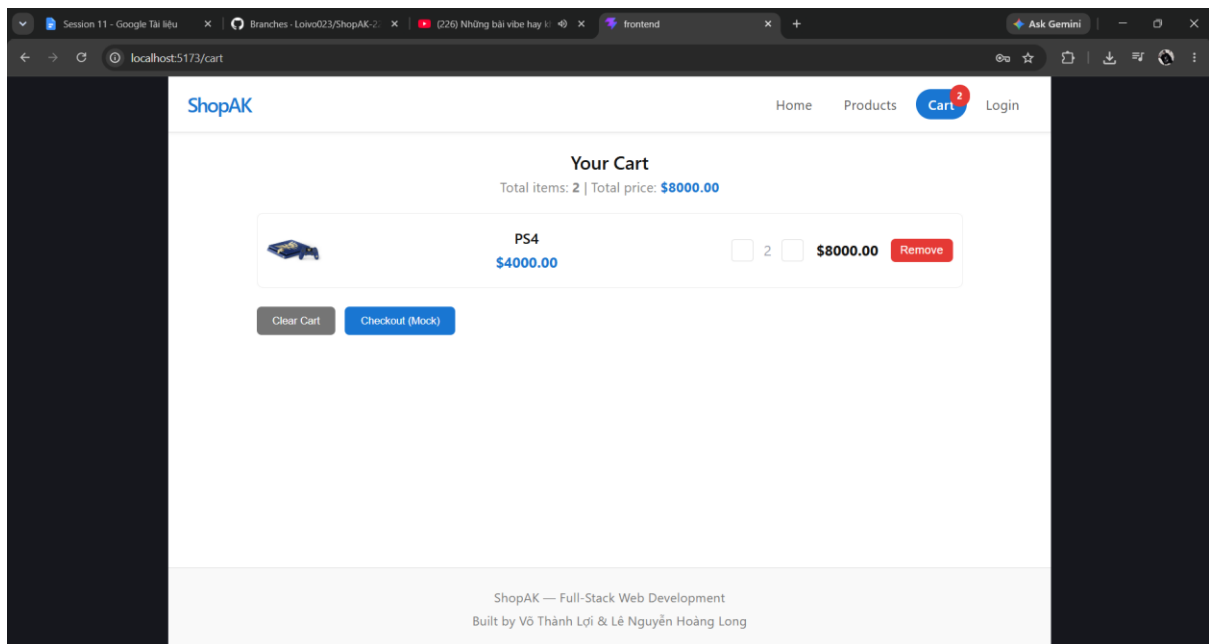
Khi click "Add to Cart" trên 1 sản phẩm nút sản phẩm sẽ thành đổi thành "✓ Added" vào giỏ hàng



Vào chi tiết 1 sản phẩm click "Add to Cart" và nút đổi màu xanh "✓ Added to Cart" và link "View Cart →" xuất hiện sau khi bấm vô sẽ đưa mình vào giỏ hàng nơi mình đã đặt hàng sản phẩm mình muốn

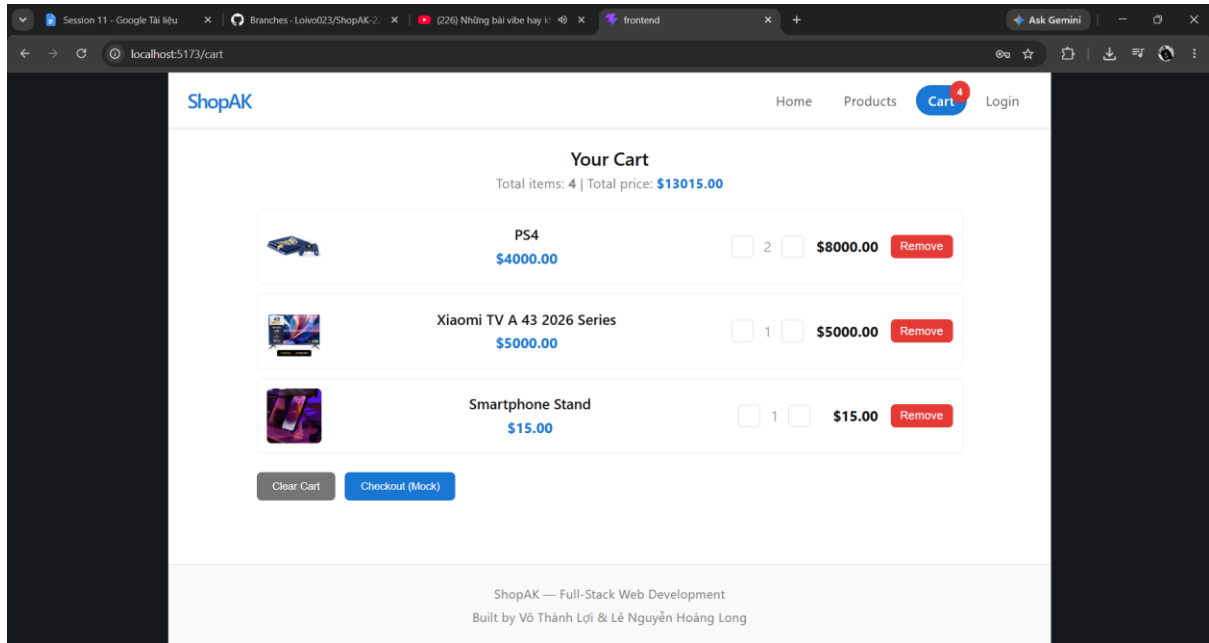


Kết quả bên trong giỏ hàng (Cart)

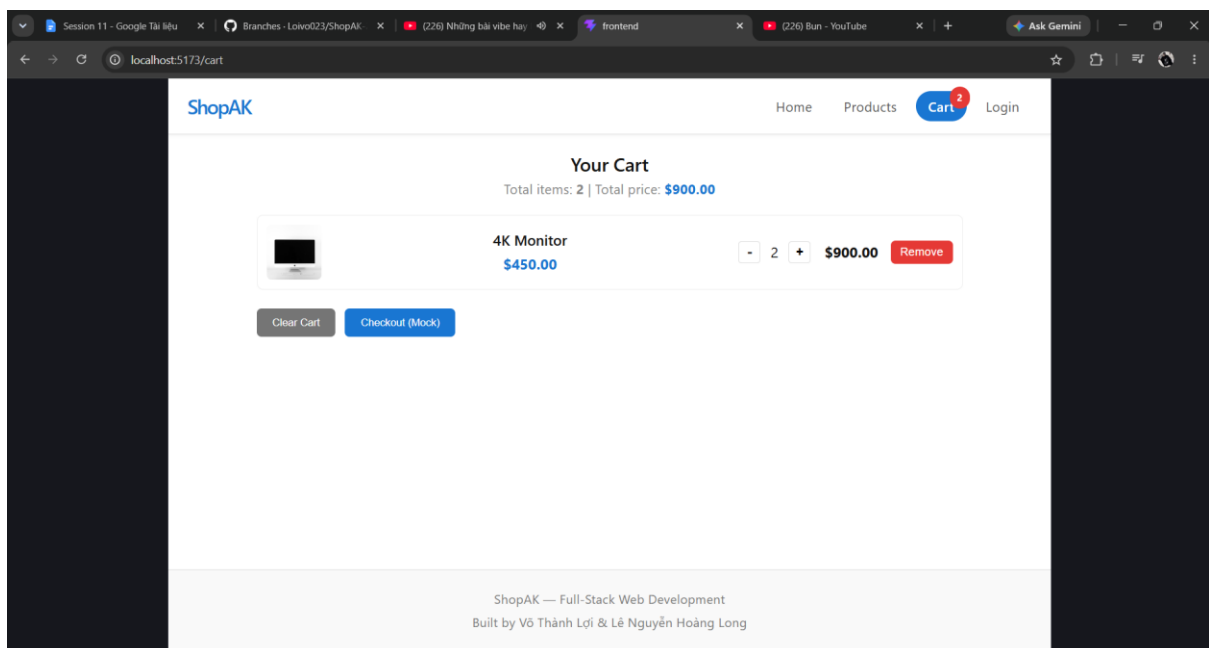


3. Build CartPage

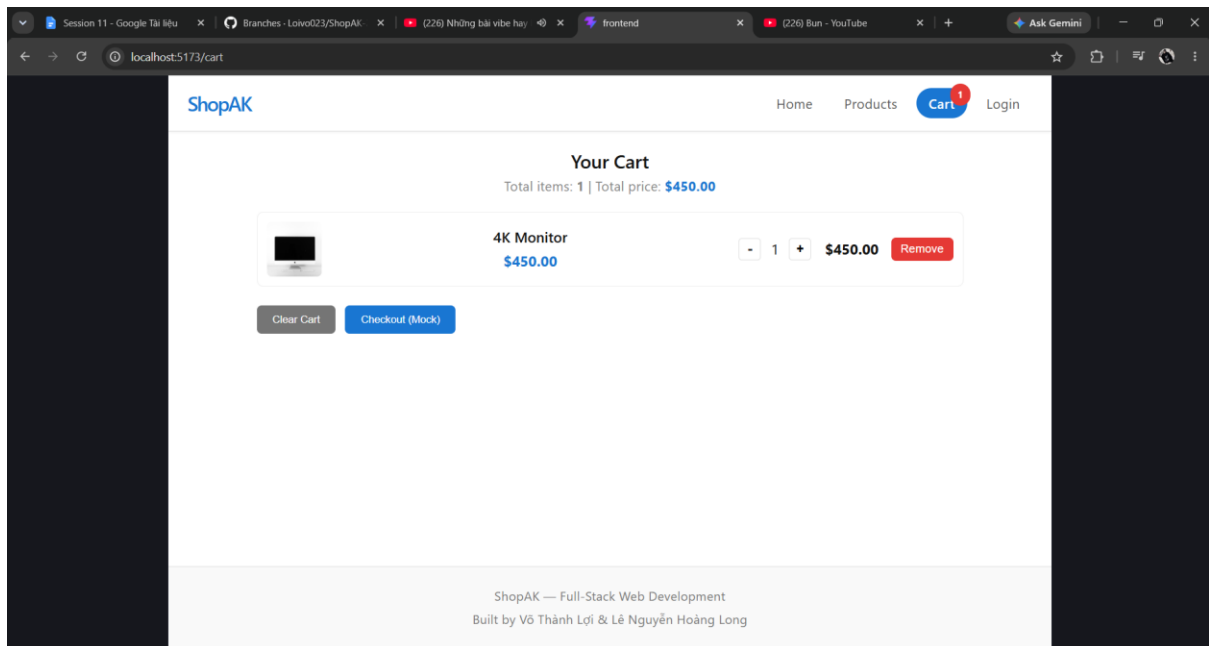
a. Hiện thị danh sách item trong giỏ với: Ảnh, tên, price, quantity, subtotal



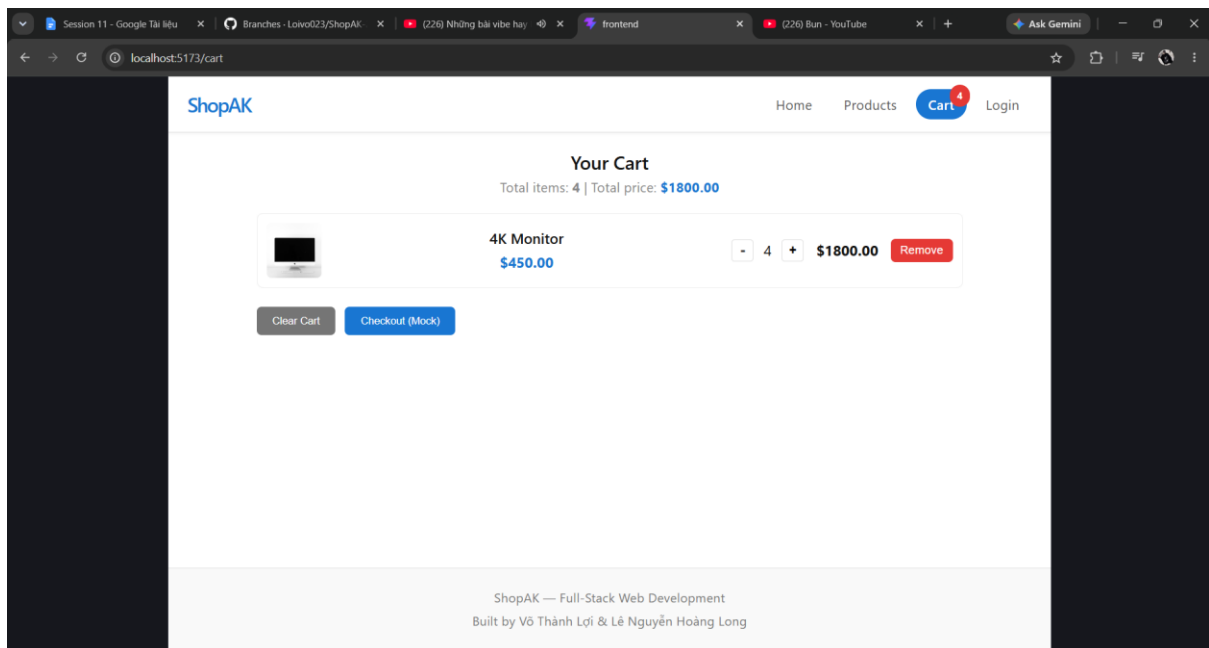
b. Implement nút + / - để gọi updateQuantity.



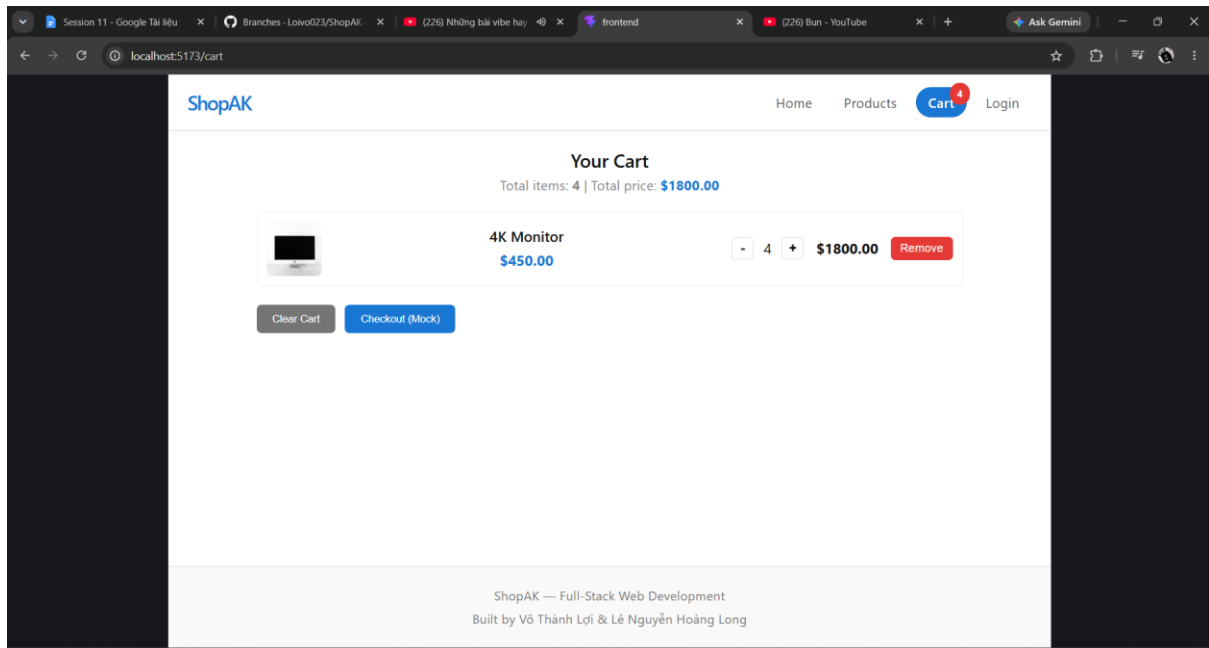
Trước khi bấm dấu – sẽ thấy quantity và subtotal thay đổi còn lại 1



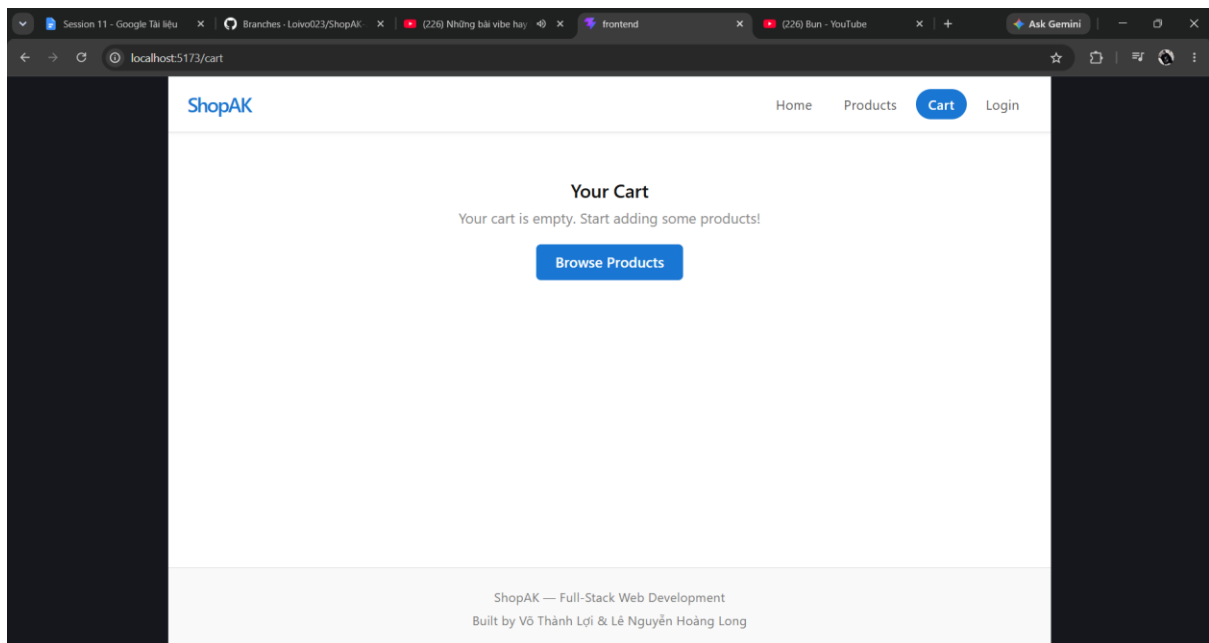
Tương tự sau khi bấm dấu + khoảng 3 lần quantity và subtotal thay đổi thành 4



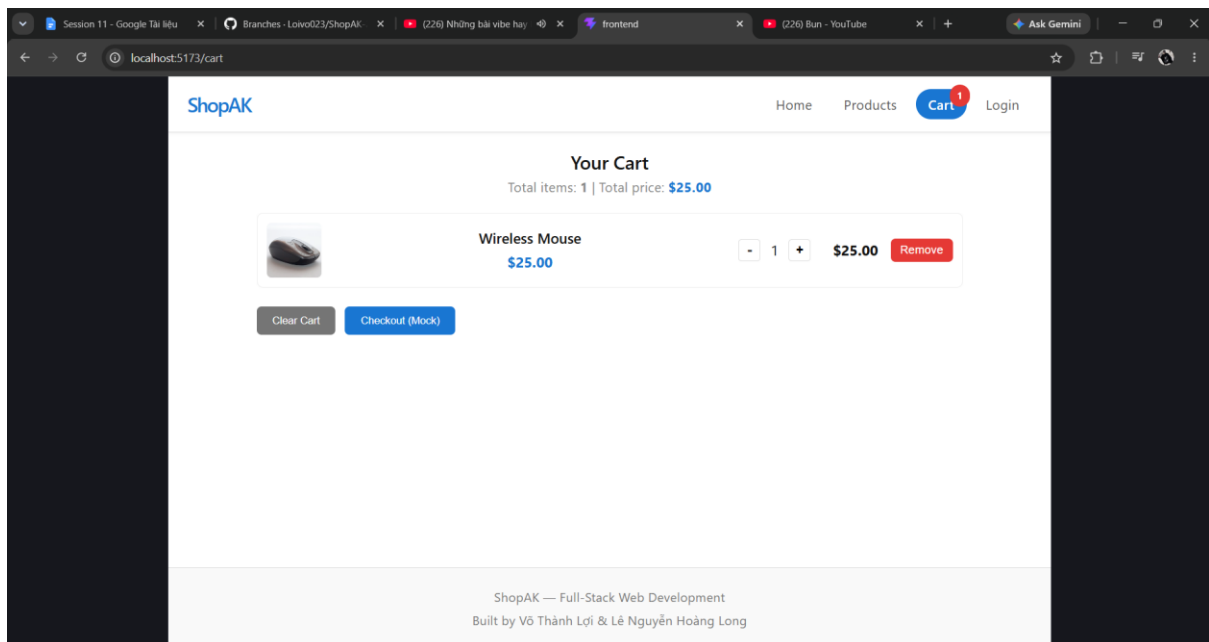
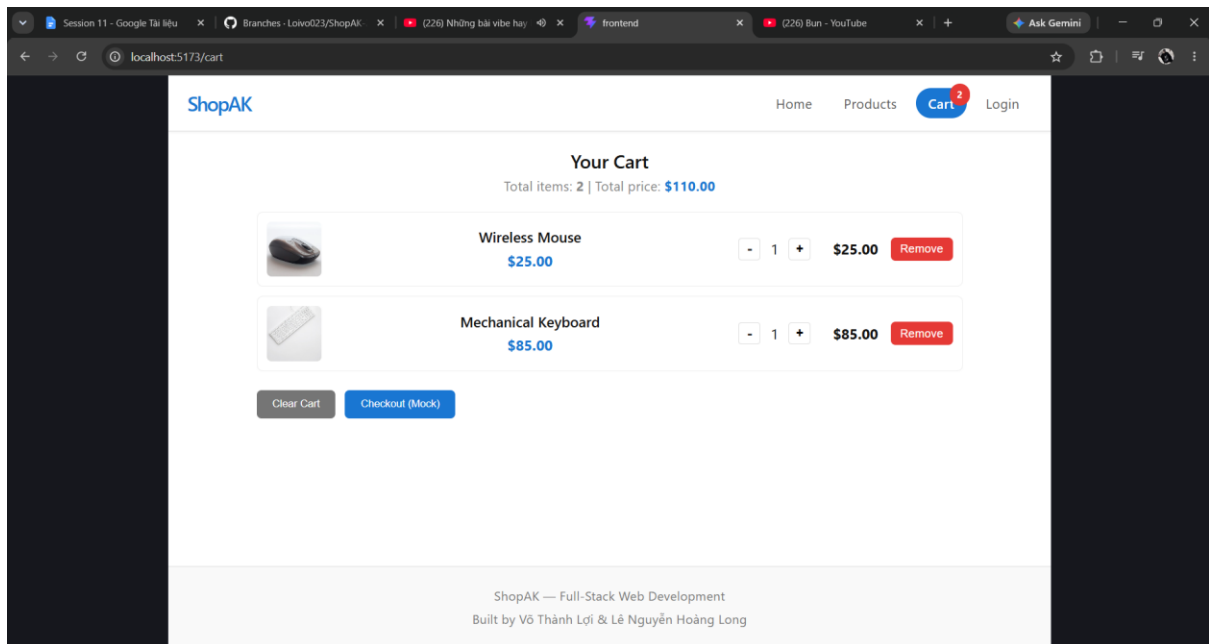
c. Implement nút Remove để gọi removeFromCart.



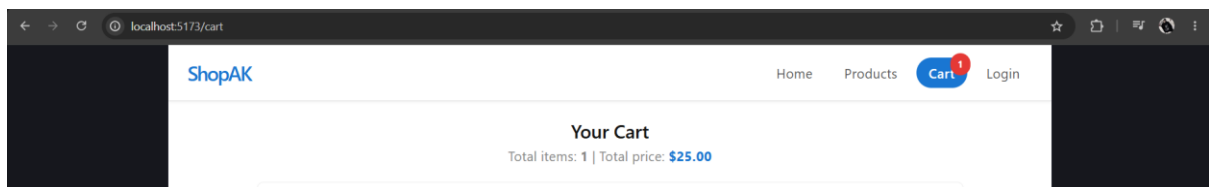
Sau khi bấm nút REMOVE sẽ thấy sản phẩm đã bị xóa biến mất khỏi danh sách giỏ hàng



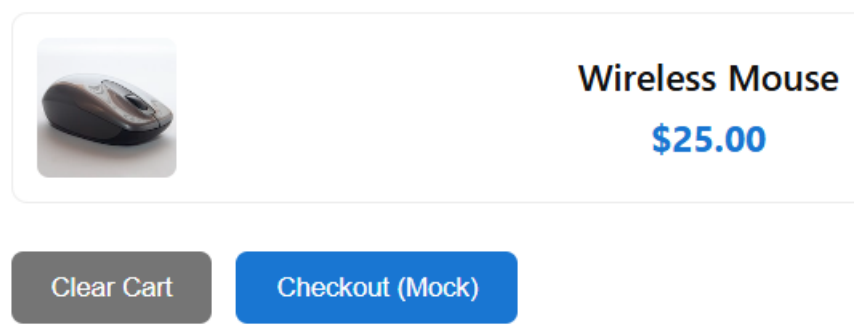
Ví dụ thêm 2 sản phẩm vào REMOVE sản phẩm thứ 2 sẽ xóa sản phẩm thứ 2



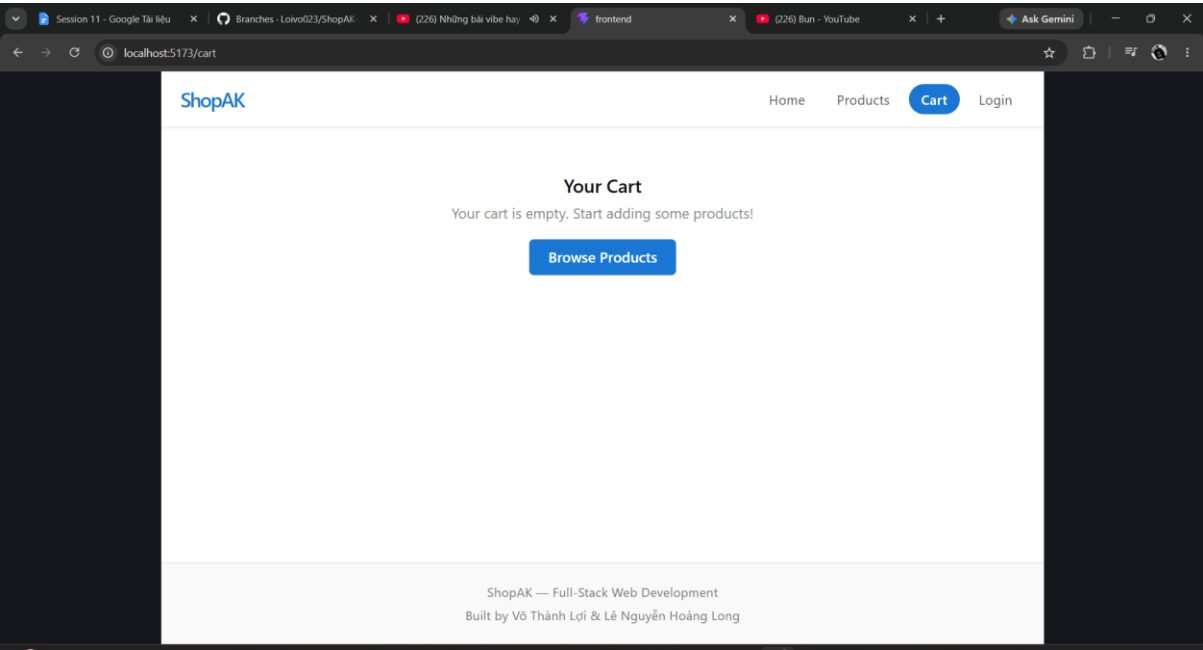
d. Hiển thị totalQuantity và totalPrice.



e. Thêm nút Clear Cart và nút Checkout (Mock).

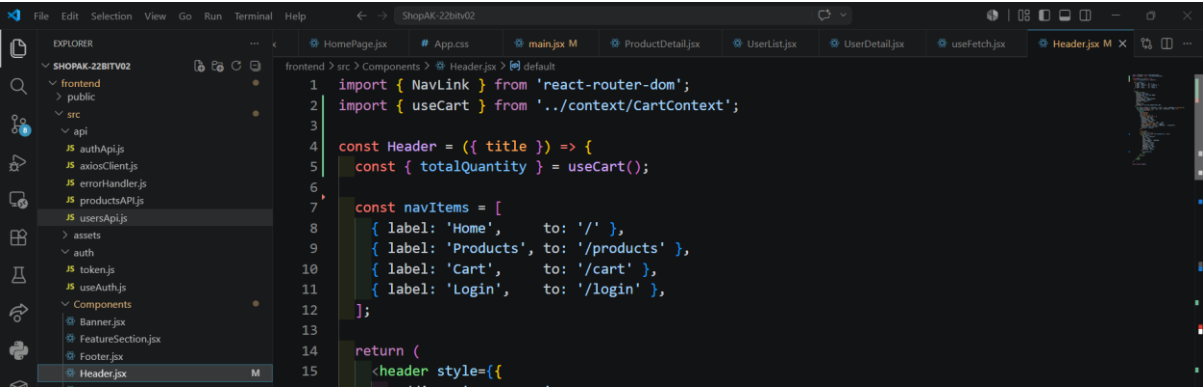


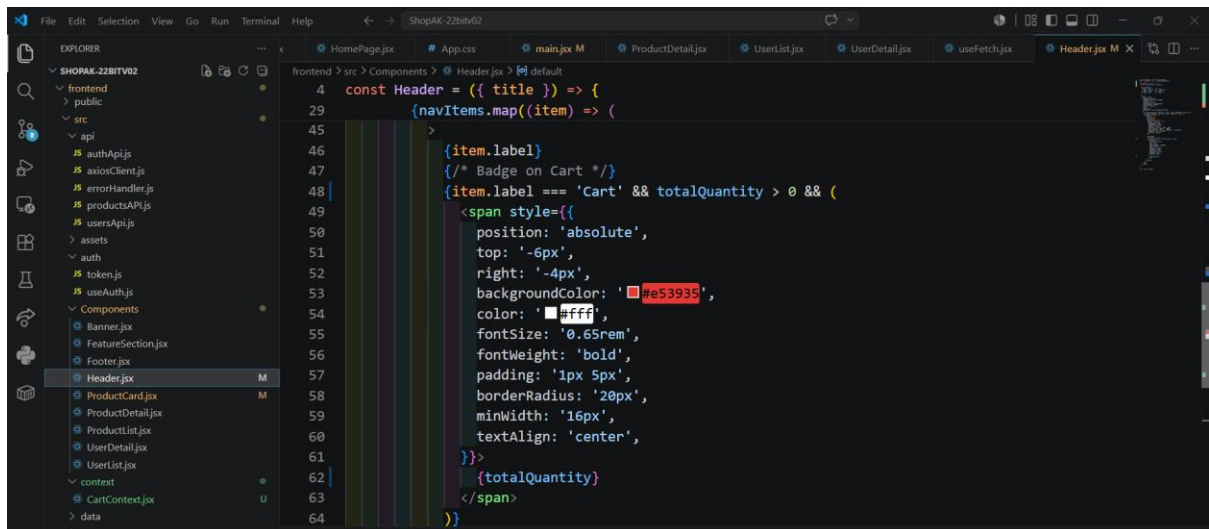
Sau khi bấm nút Clear Cart sẽ xóa sản phẩm và trang chuyển về trạng thái “Your cart is empty”



4. Cart Icon in Header

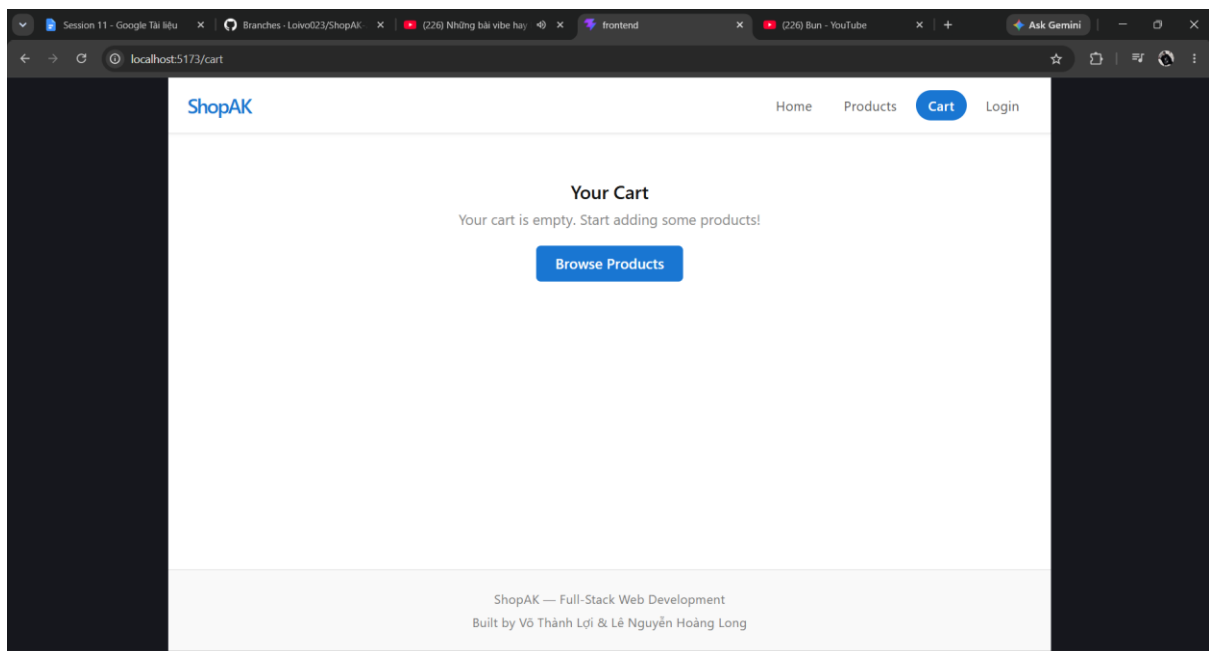
a. Hiện thị “Cart (X)” trong header, với X = totalQuantity.



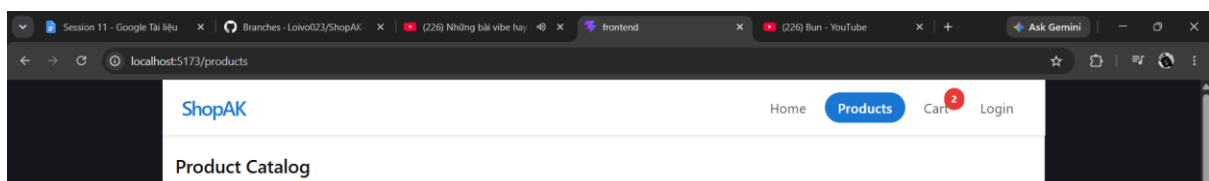


```
4 const Header = ({ title }) => {
29   {navItems.map((item) => (
45     >
46     {item.label}
47     {/* Badge on Cart */}
48     {item.label === 'Cart' && totalQuantity > 0 && (
49       <span style={{
50         position: 'absolute',
51         top: '-6px',
52         right: '-4px',
53         backgroundColor: '#e53935',
54         color: 'white',
55         fontSize: '0.65rem',
56         fontWeight: 'bold',
57         padding: '1px 5px',
58         borderRadius: '20px',
59         minWidth: '16px',
60         textAlign: 'center',
61       }}>
62         {totalQuantity}
63       </span>
64     )}
65   )}
66 }
```

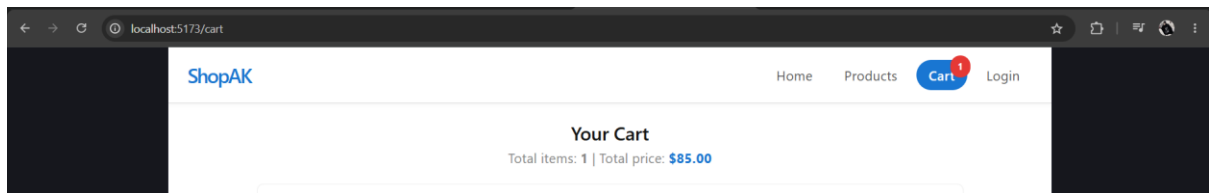
b. Khi thêm/xóa item, số lượng trên header tự động cập nhật.
header lúc giỏ hàng trống không có badge số



Sau khi thêm 1-2 sản phẩm vào giỏ chụp header ngay sau đó thấy badge đỏ hiện số lượng **mà không cần reload trang chứng tỏ Context hoạt động real time**



Sau khi Xóa sản phẩm sẽ thấy badge đỏ giảm theo sản phẩm đã xóa



Câu hỏi giải đáp

Tại sao dùng Context API thay vì truyền props qua nhiều cấp (prop drilling)?

Trước khi có Context, giỏ hàng phải được quản lý ở component gốc (App.jsx) rồi truyền xuống qua props tới từng component con như Header, ProductList, ProductDetailPage, CartPage. Khi cây component sâu, việc truyền props qua nhiều tầng trung gian không sử dụng dữ liệu đó (prop drilling) làm code khó đọc và khó bảo trì.

Với CartContext, bất kỳ component nào cũng có thể gọi useCart() để đọc hoặc cập nhật giỏ hàng trực tiếp, không cần truyền qua props trung gian. Khi state trong CartProvider thay đổi (ví dụ thêm sản phẩm), mọi component đang subscribe vào context như Header hiển thị badge số lượng sẽ tự động re-render với dữ liệu mới, tạo ra trải nghiệm real time mà không cần logic đồng bộ thủ công.